

4WD MECANUM

Contents

1. はじめに.....	3
2. 開発.....	4
2.1 Mecanum 4WD	5
2.2 Inter-Integrated Circuit (I ² C)	7
2.2.1 i2c-tools.....	8
2.3 その場回転の仕組み.....	11
2.4 使用したツール.....	13
2.4.1 クライアント (Client).....	13
2.4.2 サーバー (Server).....	14
2.4.3 Makefile.....	15
2.4.4 SSH (Secure Shell)	17
3. 結論.....	19

1. はじめに

本プロジェクトの目的は、The Linux Programming Interface を参考にしながら、Mecanum 4WD のすべてのコンポーネントを C 言語によって低レベルで制御することである。C 言語を用いてシステムを実装することで、Linux オペレーティングシステムがどのように動作しているのかを深く理解し、多様なシステムに対応できる開発能力を向上させることを目指す。

Mecanum 4WD は、もともと Python を用いて開発されたプロジェクトである。

Python は、シンプルさや高い移植性など多くの利点を持つ一方で、ハードウェアをより低レベルで制御するためには、メモリ操作やポインタを扱うことができる C 言語の利用が重要となる。

本プロジェクトの主な目的は、C 言語の各機能そのものを詳細に解説することではなく、将来的な機能拡張の基盤となるプロトタイプを構築することである。

このようなアプローチにより、本プロジェクトで得られた知識や技術は、ロボットアーム、ドローン、ステッピングモーター制御など、さまざまな組み込みシステムやハードウェア開発に応用可能な参考資料として活用できる。

2. 開発

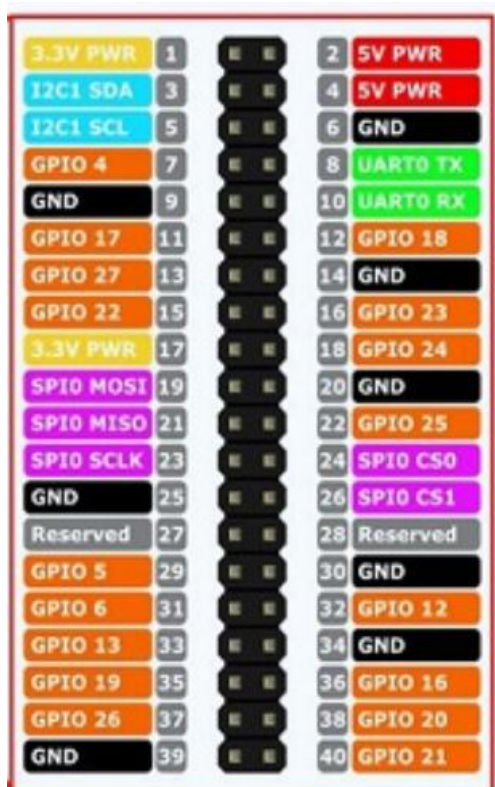
本資料を読むにあたり、読者はあらかじめ Raspberry Pi 5 に Raspberry Pi OS（旧 Raspbian）をインストールし、基本的な設定を完了しているものとする。

本章では、使用するいくつかのコンポーネントについて、それぞれの仕様書を示し、型番、バージョン、通信バス、マッピングされたアドレス、そして各デバイスのデータシートを確認する。

なお、教育上の理由により、カメラ、超音波センサー、およびサーボモーターについては本資料では扱わない。

PCA9685

Raspberry Pi 5（GPIO）



3.3V PWR	1		2	5V PWR
I2C1 SDA	3		4	5V PWR
I2C1 SCL	5		6	GND
GPIO 4	7		8	UART0 TX
GND	9		10	UART0 RX
GPIO 17	11		12	GPIO 18
GPIO 27	13		14	GND
GPIO 22	15		16	GPIO 23
3.3V PWR	17		18	GPIO 24
SPI0 MOSI	19		20	GND
SPI0 MISO	21		22	GPIO 25
SPI0 SCLK	23		24	SPI0 CS0
GND	25		26	SPI0 CS1
Reserved	27		28	Reserved
GPIO 5	29		30	GND
GPIO 6	31		32	GPIO 12
GPIO 13	33		34	GND
GPIO 19	35		36	GPIO 16
GPIO 26	37		38	GPIO 20
GND	39		40	GPIO 21

サイトから引用した画像：

<https://www.richardelectronics.com/blog/projects/raspberry%20pi/raspberry-pi-5-pinout-specs-datasheet-projects>

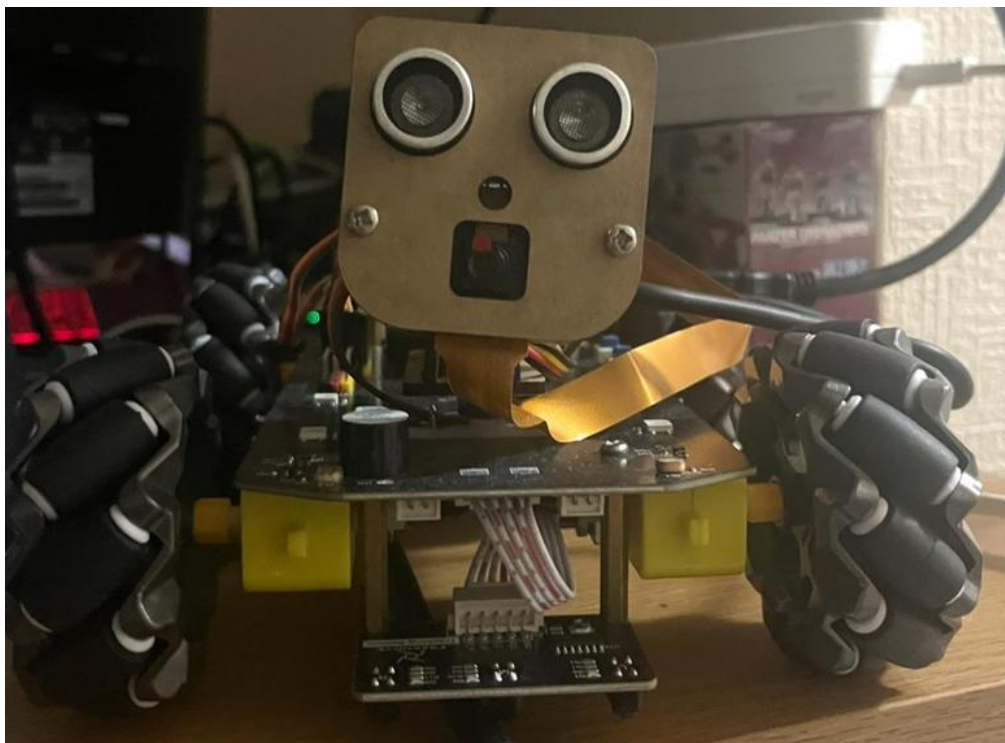
2.1 Mecanum 4WD

Freenove 4WD Smart Car Kit for Raspberry Pi は、4 輪駆動方式を採用したリモート操作可能な車両であり、その場で回転することができる。

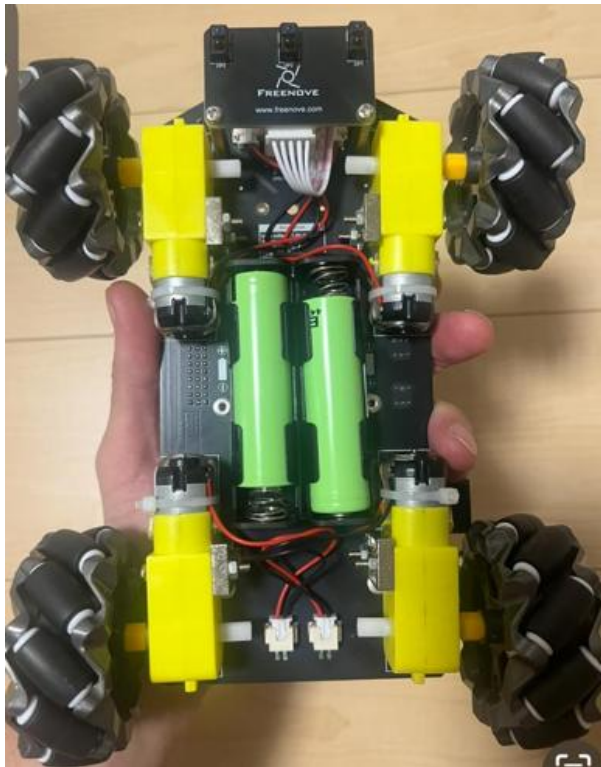
この製品は、もともと Python で制御されるように設計されており、以下のリポジトリに含まれるファイルをインストールして設定することで使用できる。

[Freenove 4WD Smart Car Kit for Raspberry Pi GitHub Repository](#)

一般的に、この車両にはカメラ、距離センサー、車体下部に取り付けられた穴や段差を検出するためのセンサー、および各デバイスを制御するためのマイクロプロセッサが搭載されている。



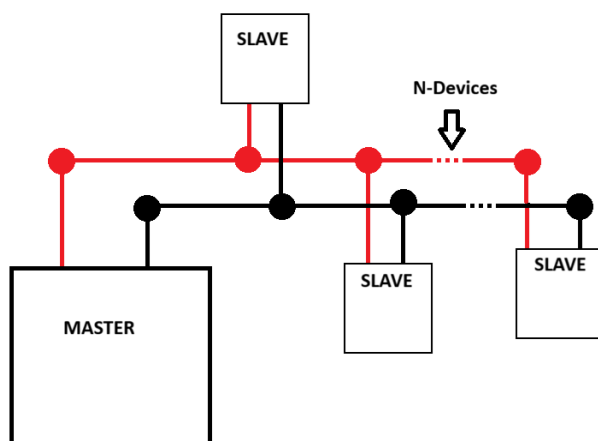




2.2 Inter-Integrated Circuit (I²C)

Inter-Integrated Circuit (I²C) は、複数のデバイス間で通信を行うためのシリアル通信プロトコルである。1 台または複数の制御側デバイス (Master) が、1 台または複数の制御対象デバイス (Slave) に対してコマンドを送信することができる。

この通信プロトコルは、必ずしも最も高速な方式ではないが、SDA (Serial Data Line) と SCL (Serial Clock Line) の 2 本の信号線のみを用いて、最大 127 台のデバイスを接続できるという特徴を持つ。



2.2.1 i2c-tools

i2c-tools に含まれるツールを使用することで、特定の I²C バスへアクセスし、I²C 通信の状態を確認することができる。

接続されている I²C バスを一覧表示するには、以下のコマンドを使用する。

i2cdetect -l

```
vic@raspberrypi:~ $ i2cdetect -l
i2c-1  i2c      Synopsys DesignWare I2C adapter      I2C adapter
i2c-10 i2c      Synopsys DesignWare I2C adapter      I2C adapter
i2c-13 i2c      107d508200.i2c      I2C adapter
i2c-14 i2c      107d508280.i2c      I2C adapter
vic@raspberrypi:~ $
```

この例では、I²C バスは「1」であり、i2c-1 として識別されている。

通常、10 以上の番号が割り当てられているバスは、システム内部で使用する標準的な通信インターフェースに関連していることが多い。

接続されている Slave デバイスのアドレスを確認するには、以下のコマンドを使用する。

i2cdetect -y 1

```
vic@raspberrypi:~ $ i2cdetect -y 1
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: 40 -- -- -- -- -- -- -- 48 -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: 70 -- -- -- -- -- -- -- -- -- -- -- -- --
```

引数 -y は、確認メッセージを表示せずにコマンドを実行するためのオプションであり、ユーザーが操作内容を理解したうえで手動で実行していることを示す。

現在使用しているチップは、PCA9685 である。公式データシートによると、I²C アドレス 0x40 が既定の通信アドレスとして設定されており、このアドレスを通じて車輪、カメラ、サーボモーター、さらに必要に応じてその他のセンサーを制御することができる。

このアドレスに割り当てられたレジスタを確認するには、以下のコマンドを使用する。

```
i2cdump -y 1 0x40
```

```
vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
  0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f  0123456789abcdef
00: 11 04 e2 e4 e8 e0 00 00 00 10 00 00 00 10 00 00  ??????...?..?..
10: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00  .?...?...?...?..
20: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00  .?...?...?...?..
30: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00  .?...?...?...?..
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX  .?...?XXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX XX 00 00 00 00 1e 00  XXXXXXXXXXX....?.
```

この画像では、16 行×16 列の表が表示されており、設定されているレジスタの内容を確認することができる。今回の例では、0x00 から 0x44 までの合計 70 個のレジスタが使用されている。

表の見方として、行は上位 1 桁（16 の位）、列は下位 1 桁（1 の位）を表している。

例えば、行 0x50 と列 0x05 の交点は、レジスタ 0x55 を示す。

各レジスタには 16 進数の値が表示されており、それぞれのレジスタに現在格納されているデータを表している。

外部デバイスが正常に認識されている場合、Raspberry Pi 5 はアクセス可能なレジスタの内容を表示する。一方、アクセスできないレジスタについては XX と表示される。

注記：

各ハードウェアは外部電源またはバッテリーによって駆動されている。これらの電源供給が停止すると、I²C デバイスとの通信ができなくなり、表内のすべてのレジスタが XX と表示される。

```

vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
10: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
20: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
30: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
40: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX

```

i2c-tools の使用例および各種コマンドについては、以下の GitHub リポジトリにまとめている。

<https://github.com/NagaoVictor/i2c/blob/main/new%2016.txt>

特定のレジスタに格納されている 1 バイトの値を読み取るには、i2cget コマンドを使用する。

```
i2cget -y 1 0x40 0x06
```

上記の例では、I²C バス 1 に接続されたアドレス 0x40 のデバイスに対して、レジスタ 0x06 の値を読み取っている。

一般的な書式は以下のとおりである。

```
i2cget -y <bus> <address> <register>
```

```

vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 11 04 e2 e4 e8 e0 00 00 00 10 00 00 00 10 00 00  ?????.?..?..
10: 00 10 00 00 00 00 10 00 00 00 10 00 00 00 10 00  .?..?..?..?..
20: 00 10 00 00 00 00 10 00 00 00 10 00 00 00 10 00  .?..?..?..?..
30: 00 10 00 00 00 00 10 00 00 00 10 00 00 00 10 00  .?..?..?..?..
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX  .?..?XXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX XX 00 00 00 1e 00  XXXXXXXXXX....?.
vic@raspberrypi:~ $ i2cget -y 1 0x40 0x01
0x04

```

特定のレジスタに1バイトの値を書き込むには、i2cset コマンドを使用する。

```
i2cset -y <bus> <address> <register> <value>
```

例えば、I²C バス 1 に接続されたアドレス 0x40 のデバイスのレジスタ 0x06 に値 0x00 を書き込む場合は、以下のように実行する。

```
i2cset -y 1 0x40 0x06 0xFF
```

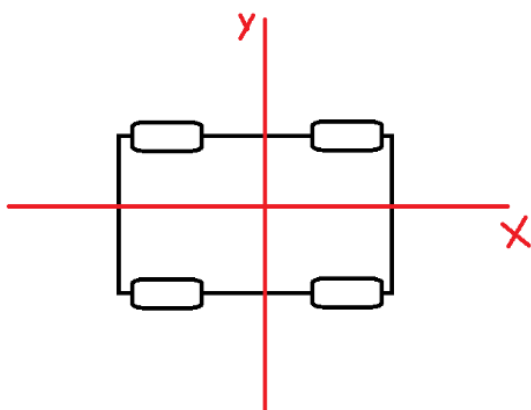
```
vic@raspberrypi:~ $ i2cset -y 1 0x40 0x06 0xFF
vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 11 04 e2 e4 e8 e0 ff 00 00 10 00 00 00 10 00 00    ??????...?...?..
10: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
20: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
30: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX    .?...?XXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX 00 00 00 00 1e 00    XXXXXXXXXXX....?.
```

上記の例では、値 0xFF がレジスタ 0x06 に書き込まれている。

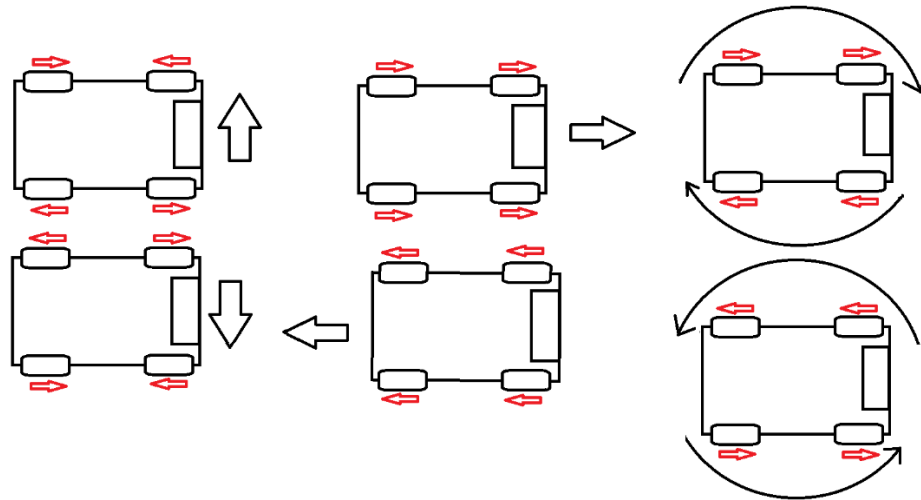
2.3 その場回転の仕組み

メカナムホイールを搭載した車両は、一般的な自動車のように前輪の向きを変えるためのステアリング機構を持たない。

そのため、方向転換を行う際には、各車輪に取り付けられたローラーが特定のパターンで回転することで、車体の移動方向や回転方向を制御する。これにより、車両はその場で回転したり、横方向へ移動したりすることが可能となる。



現時点では、6種類の異なる動作がプログラムされている。



各チャンネルはそれぞれ異なる周波数で動作するため、ハードウェアの動作に対して送信するコマンドの伝送速度を補正する必要がある。

この補正係数を s とし、`apply_mecanum` メソッド内の各式に適用する。

```
void drive_bridge(int chA, int chB, int speed){
    if (speed>=0){
        set_pwm(chA, speed > 4095 ? 4095 : speed);
        set_pwm(chB,0);
    } else {
        set_pwm(chA, 0);
        set_pwm(chB, -speed > 4095 ? 4095 : -speed);
    }
}
```

キーボードから入力されるコマンドは、X 軸方向の移動量を表す x 、Y 軸方向の移動量を表す y 、および回転を表す r の値を決定する。

```
if(tecla=='w'){cmd.y=100;cmd.x=0; cmd.r=0;}
else if(tecla=='s'){cmd.y=-100;cmd.x=0; cmd.r=0;}
else if(tecla=='a'){cmd.y=0;cmd.x=-100;cmd.r=0;}
else if(tecla=='d'){cmd.y;cmd.x=100;cmd.r=0;}
else if(tecla=='j'){cmd.y=0; cmd.x=0;cmd.r=-50;}
else if(tecla=='l'){cmd.y=0;cmd.x=0;cmd.r=50;}
else if(tecla=='q')break;
else {cmd.y =0; cmd.x=0; cmd.r=0; }
```

```
void apply_mecanum(int8_t x, int8_t y, int8_t r){
    ioctl(fd_i2c, I2C_SLAVE, PCA9685_ADDR);

    int s = 32;

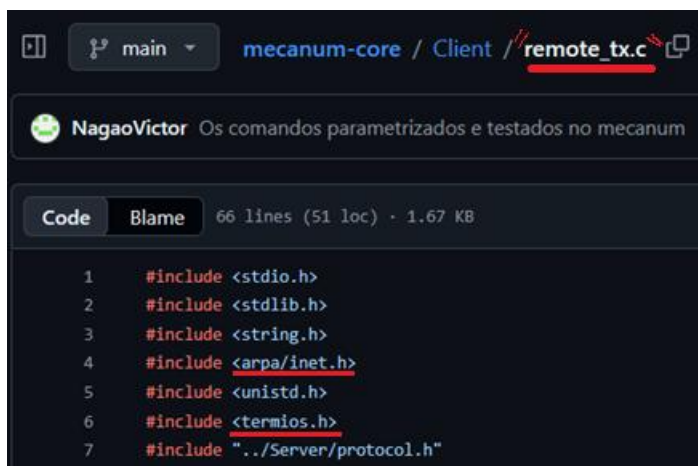
    int fl = (y + x + r) * s;
    int rl = (y - x + r) * s;
    int rr = (y + x - r) * s;
    int fr = (y - x - r) * s;

    drive_bridge(0, 1, fl);
    drive_bridge(3, 2, rl);
    drive_bridge(4, 5, rr);
    drive_bridge(6, 7, fr);
}
```

2.4 使用したツール

```
/home/pi/mecanum_core/
├── Server/                                # Roda no Raspberry Pi 5 (0 Core)
│   ├── Makefile                         # Compilação otimizada para ARM Cortex-A76
│   ├── main.c                           # Orquestrador (Loop de Controle)
│   ├── motor_driver.c/.h                # I2C Nativo (Conversa com o PCA9685)
│   ├── network_mgr.c/.h                 # Sockets UDP via Tailscale (Interface tun0)
│   └── protocol.h                       # O "Contrato" entre PC e Robô
├── Client/                              # Roda no seu PC (O Painel)
│   ├── protocol.h                       # O "Contrato" entre PC e Robô
│   └── remote_tx.c                      # Envia comandos para o IP do Tailscale
└── Itens gerais (Imagens, Textos, ...)
```

2.4.1 クライアント (Client)



The screenshot shows a code editor interface. At the top, there's a breadcrumb navigation: `mecanum-core / Client / remote_tx.c`. Below it, a message from NagaoVictor says: "Os comandos parametrizados e testados no mecanum". There are tabs for "Code" and "Blame", with "Code" selected. The file statistics show "66 lines (51 loc) · 1.67 KB". The code content shows the first seven lines of `remote_tx.c`, which are all include statements.

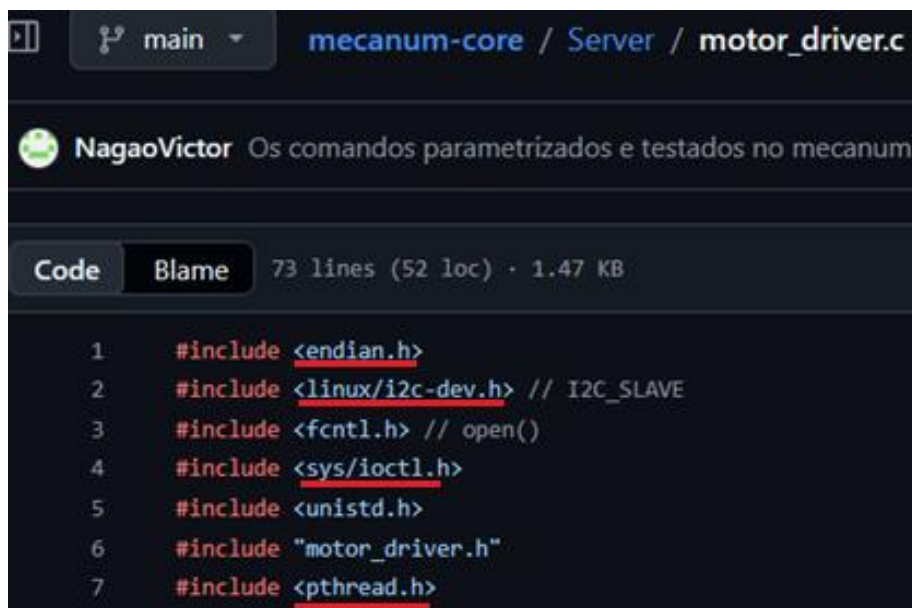
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <termios.h>
7  #include "../Server/protocol.h"
```

Link: https://github.com/NagaoVictor/mecanum-core/tree/main/Server/remote_tx.c

無線通信を実現するために、ソケット通信の関数や構造体を提供する `arpa/inet.h` ライブラリを使用し、ネットワークインターフェースを介した通信チャンネルを構築した。

また、キーボードからの入力を1文字ずつ読み取り、そのデータをネットワークへ送信するために、Universal Asynchronous Receiver-Transmitter の概念を利用した端末入出力制御を行った。これにより、各キー入力をリアルタイムで取得し、サーバーへ送信することが可能となる。

2.4.2 サーバー (Server)



The screenshot shows a GitHub repository view for the file `motor_driver.c` in the `mecanum-core` repository, specifically in the `Server` directory. The file is 73 lines long (52 loc) and 1.47 KB. The code includes the following headers:

```
1  #include <endian.h>
2  #include <linux/i2c-dev.h> // I2C_SLAVE
3  #include <fcntl.h> // open()
4  #include <sys/ioctl.h>
5  #include <unistd.h>
6  #include "motor_driver.h"
7  #include <pthread.h>
```

Link: https://github.com/NagaoVictor/mecanum-core/blob/main/Server/motor_driver.c

サーバー側では、本アプリケーションの実装に必要な複数のヘッダファイルを使用している。これらには並列処理を実現するための機能が含まれており、適切な箇所で利用することで、ロボットの動作性能を向上させるとともに、いわゆる Race Condition（競合状態）の発生を防ぐことができる。

`endian.h` は、システムごとに異なるバイト順（エンディアン）を調整し、データを正しい順序で解釈するための機能を提供する。

`sys/ioctl.h` は、デバイスファイルに対して制御命令を送信し、順序付けられたバイト列を用いてハードウェアを操作するためのインターフェースを提供する。

linux/i2c-dev.h は、Linux 上で I²C 通信を行うための定義を提供し、Raspberry Pi 5 と車両のハードウェアとの通信を可能にする。

最後に、POSIX Threads (pthread) は、ソフトウェアおよび各デバイスの並列処理を実現するためのライブラリである。これにより、1つのプログラムの実行中に複数のスレッドを同時に動作させ、互いに独立した処理や連携した処理を並行して実行することができる。



```
main ▾ mecanum-core / Server / network_mgr.c
NagaoVictor Infra: configuracao inicial e ganchos de hardware
Code Blame 22 lines (20 loc) · 519 Bytes
1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <netinet/in.h>
4  #include <arpa/inet.h>
5  #include "protocol.h"
6
```

Link: https://github.com/NagaoVictor/mecanum-core/blob/main/Server/network_mgr.c

これらのライブラリを組み合わせることで、Raspberry Pi 5 のネットワーク機能を利用し、クライアントからの接続を受け付けることができる。

sys/socket.h、netinet/in.h、および arpa/inet.h は、クライアントとの通信を受信するためのソケット通信機能を構築する際に使用される主要なライブラリである。

2.4.3 Makefile

GNU Make の Makefile は、実行することで拡張子 .c のソースファイルをまとめてコンパイルするための設定ファイルである。

Linux 上で実行権限を付与し、その後に Makefile を実行するには、以下のコマンドを使用する。sudo chmod a+x makefile && ./makefile



```
1 CC = gcc
2 CFLAGS = -Wall -O3 -march=armv8.2-a+crypto
3 LDFLAGS = -lpthread
4
5 mecanumbot: main.c network_mgr.c motor_driver.c
6 $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)
7
8 clean:
9 rm -f mecanumbot
10 sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Link: <https://github.com/NagaoVictor/mecanum-core/blob/main/Server/Makefile>

上の図には、Makefile ファイルの内容を示している。

クライアント側では、コンパイル対象となるソースファイルが1つのみであるため、Makefile は使用していない。そのため、GNU Compiler Collection (gcc) を用いて、以下のコマンドでコンパイルおよび実行を行う。

```
sudo chmod a+x remote_tx.c && gcc remote_tx.c -o cc && ./cc
```

上記のコマンドは、ソースファイルに実行権限を付与し、コンパイルを行った後、生成された実行ファイルを連続して実行する。

```
vic@vbox:~/mecanum_core/Client$ sudo chmod a+x remote_tx.c && gcc remote_tx.c -o cc && ./cc
[sudo] password for vic:
Controle Remoto Ativo via Tailscale. Pressione W para mover...
Enviado -> Y:-100 X:0 R:0
```

上の図では、一般的なノートパソコンを使用し、キーボードを通じて車両を操作する。

```
vic@raspberrypi:~/mecanum_core/Server $ sudo chmod a+x Makefile && ./mecanumbot
--- Core do Robo Ativo (PI 5) ---
[Y: 0 | X: 0]
```

別の図では、Mecanum 4WD に搭載された Raspberry Pi 5 上でサーバープログラムが動作している。

クライアント側でコマンドを入力すると、C 言語のネットワークライブラリが動作し、TCP/IP 通信を通じてサーバーへ信号を送信する。

なお、本プロジェクトではソケット通信を使用しているため、通信方式は Transmission Control Protocol に基づいており、OpenSSH (SSH) そのものを使用しているわけではない。

2.4.4 SSH (Secure Shell)

OpenSSH (SSH: Secure Shell) は、遠隔地にあるデバイスへ安全に接続するための通信プロトコルである。

SSH を使用することで、無線ネットワーク、有線ネットワーク、仮想マシン環境など、さまざまな環境において暗号化された通信を確立することができる。

SSH を利用するには、接続先のシステムで SSH サーバーを有効化し、起動する必要がある。以下のコマンドを実行することで設定できる。

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

```
vic@vbox:~$ sudo systemctl enable ssh
[sudo] password for vic:
Synchronizing state of ssh.service with SysV service script with /lib/systemd/sy
stemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
```

```
|vic@vbox:~$ sudo systemctl start ssh
```

SSH を有効化した後、リモートデバイスへ接続するには、以下のコマンドを使用する。

```
ssh <username>@<ip>
```

上記のコマンドを実行すると、パスワードの入力が求められるため、接続先デバイスのパスワードを入力する。

```
|vic@vbox:~/mecanum_core$ ssh vic@100.78.54.126
vic@100.78.54.126's password: █
```

注記：

上記の IP アドレスは VPN を使用しているため、接続のたびに変更される場合がある。

3. 結論

本プロジェクトは、C 言語を用いて Mecanum 4WD に機能的な制御機構を実装することを目的として構想された。

本資料で作成したモデルを基盤とすることで、既存の制御機構を容易に変更できるほか、新たなデバイスを追加して機能を拡張することも可能である。